PRPrompt: Prototype-Routed Graph Prompting for Few-Shot Node Classification

Abstract

We introduce PRPrompt, a graph prompting method that uses support-set prototypes to route trainable prompts to individual nodes. With only a few labeled nodes per class, adaptation must determine both what a prompt should encode and where its contribution should be applied. PRPrompt constructs class references from aggregated node attributes, refines them using unlabeled graph features while retaining support anchors, and converts node–prototype associations into a fixed routing matrix. The resulting combinations of class prompt tokens are injected into a frozen graph convolutional encoder, followed by prototype classification. We evaluate each 1-, 3-, and 5-shot setting separately on Cora, CiteSeer, and PubMed using the original ProG sampler, which can overlap support and test nodes, with matched task draws and training seeds. In the 5-shot setting, PRPrompt achieves accuracies of 72.87%, 64.70%, and 68.34%, respectively; replacing a shared prompt with prototype routing improves accuracy by 36.57, 39.09, and 3.34 percentage points in the corresponding conditions. These paired results support the value of prototype-guided prompt allocation within this overlapping transductive evaluation protocol.

1 Introduction

Pretrained graph neural networks offer a starting point for learning with scarce node labels. Graph prompting adapts these representations through a small set of task parameters, using feature modifications, prompt structures, or task-specific readout [3, 10, 14, 15]. Beyond choosing an injection point, an adaptation method must decide how a prompt's contribution is distributed across nodes. This allocation matters because nodes in the same graph can play different roles in the downstream classification task.

A shared prompt expresses a global adjustment. More flexible methods learn input-dependent mixtures, assign prompts to subgraphs, or select nodes through a policy [3, 8, 22]. These approaches establish that node-dependent allocation is not itself a new idea. Our question is more specific: *can the few labeled examples already available for a task define useful routing coefficients, so that optimization can focus on prompt content?* Support examples provide class references that can be constructed before adaptation and used without training an additional routing network.

We propose PRPrompt, which uses support-anchored prototypes as coordinates for prompt allocation. Nodes with similar class associations receive related combinations of class prompt tokens. The routing representation comes from aggregated graph attributes; its prototypes are initialized by labeled support nodes and refined with unlabeled features while retaining the support anchors. These associations yield a fixed routing matrix that mixes trainable tokens inside a frozen GCN. Only the prompt tokens are optimized, and prediction uses a prototype readout. Figure 1 summarizes this separation between fixed allocation and learned content.

Our evaluation asks two questions. First, how does complete adaptation compare with frozen pretraining and full fine-tuning in matched local runs, and where does its accuracy lie relative to literature-reported prompt results? Second, does prototype routing add value over a shared prompt when the injection point and training recipe are held fixed? The main text covers Cora, CiteSeer, and PubMed at 1, 3, and 5 shots; five further datasets appear in Appendix D. External accuracy values come from Chen et al. [2] and retain their original protocol. Every result remains specific to its dataset and shot count. The gain over frozen pretraining is much smaller on PubMed than on Cora or CiteSeer.

The paper makes three contributions:

- A graph prompting construction that uses support-anchored class relationships to fix node-specific allocation before prompt optimization.
- A hidden-layer implementation with $C \times 128$ trainable prompt parameters and no learned routing network.
- Matched adaptation comparisons and shared-prompt ablations, with seed variability and paired uncertainty reported separately for each evaluated condition.

The contribution is the use of prototype relationships to organize hidden-layer prompts; it does not treat prototypes or node-dependent prompting as new primitives.

2 Related Work

Graph representations and pretraining. Graph convolution, neighborhood sampling, and graph attention provide complementary ways to construct node representations [4, 7, 16]. Self-supervised objectives include local–global information maximization [17], augmentation-based contrastive learning [18], and masked reconstruction [5]. Pretraining objectives can affect downstream transfer, and combining node-

and graph-level objectives has been studied explicitly [6]. Our study fixes the pretrained checkpoint on each graph and examines downstream adaptation, rather than proposing a new pretraining objective.

Aligning pretraining with downstream tasks. GPPT and GraphPrompt connect pretraining to downstream prediction through task reformulation and prompting [10, 14]. All in One addresses multiple task levels with prompt tokens, structures, and insertion rules [15]; MultiGPrompt combines multiple pretext tasks and downstream prompts [20]. HGPrompt and HetGPT extend prompting to heterogeneous graph settings [11, 19]. These methods address task or graph-type alignment. We focus on a narrower design question within few-shot node classification: how task-class references should allocate prompts across nodes.

Adaptive prompt allocation. GPF-plus learns input-dependent prompt combinations [3], SUPT assigns prompts at subgraph level [8], and RELIEF learns node selection together with prompt content [22]. ProNoG studies conditional node-specific prompts for non-homophilic graphs [21], while DAGPrompT addresses distribution-aware adaptation [1]. Attribute-guided dynamic prompting has also been studied for robustness to graph perturbations [9]. In contrast to learning an allocation function, PRPrompt computes support-conditioned prototype relationships once and keeps the resulting route fixed. These works delimit the design space; they are not all experimental baselines in our controlled comparison.

Class prototypes and evaluation. Prototypical Networks use support-set class representatives for prediction [13], and semi-supervised extensions incorporate unlabeled examples into prototype construction [12]. We use these established ideas to obtain task references for routing. Routing prototypes live in aggregated attribute space; prediction prototypes come from the adapted encoder output. ProG [23] motivates explicit evaluation protocols. Chen et al. [2] report adaptation results across pretraining strategies; we reuse their baseline accuracy values as literature context while retaining local matched controls for the routing claim.

3 Problem Setting

Let $G = (V, E, X)$ have n nodes, features $X \in \mathbb{R}^{n\times d}$, and C classes. A support set S_c contains k_c labeled nodes, with nominal shot count $k \in \{1, 3, 5\}$. The three main datasets satisfy $k_c = k$; the appendix records class shortages. Let $S = \bigcup_c S_c$ and let T be the test set produced by the original ProG implementation, allowing $S \cap T \neq \varnothing$. Full topology and features are visible. Fitting and selection receive labels in S, and scoring follows prediction storage. Overlapping test nodes have already supplied training labels; these scores are therefore not strictly held-out generalization measurements.

The encoder is a pretrained two-layer GCN [7] with hidden and output dimensions of 128. Its parameters are frozen for PRPrompt. The task is transductive, same-graph classification over known classes: query features participate in graph aggregation, prototype refinement, and global centering. It is not an inductive unseen-class evaluation. Throughout the paper, a *condition* fixes the dataset, shot count, method, and pretrained checkpoint.

4 Prototype-Routed Prompting

Overview. The method separates allocation from content. First, support-anchored prototypes define class relationships in aggregated attribute space. Second, a fixed routing matrix turns those relationships into node-specific combinations of prompts. Third, support supervision learns the prompt tokens inside the frozen encoder. The following subsections follow this computation order.

4.1 Support-anchored class references

Let $\widehat{A} = \widetilde{D}^{-1/2}(A + I)\widetilde{D}^{-1/2}$ denote the normalized adjacency with self-loops. We form a routing representation from two-hop aggregated attributes:

$$\begin{aligned} X^{(2)} &= \widehat{A}^2 X, \\ R &= \mathrm{rownorm}\Big(X^{(2)} - \mathbf{1}\ \mathrm{mean}_{v\in V}\, X_v^{(2)}\Big). \end{aligned} \tag{1}$$

Here rownorm applies Euclidean normalization to each row. Routing therefore uses graph attributes and structure, rather than a separate learned network. Initial class references are

$$\mu_c^{(0)} = \frac{1}{k_c}\sum_{i\in S_c} R_i. \tag{2}$$

To incorporate the unlabeled graph while retaining a labeled reference, let $U_V = V \setminus S$. Starting from $\mu^{(0)}$, we use $i \in U_V$ and $t = 0, 1, 2$ for three fixed refinement iterations:

$$a_{ic}^{(t)} = \frac{\exp(R_i^\top \mathrm{norm}\Big(\mu_c^{(t)}\Big)/0.05)}{\sum_{c'}\exp(R_i^\top \mathrm{norm}\Big(\mu_{c'}^{(t)}\Big)/0.05)}, \tag{3}$$

$$\mu_c^{(t+1)} = 0.5\mu_c^{(0)} + 0.5\frac{\sum_{i\in U_V} a_{ic}^{(t)} R_i}{\max(\sum_{i\in U_V} a_{ic}^{(t)}, 10^{-8})}. \tag{4}$$

The anchor is always the original support prototype, not the previous iterate. This is a fixed prototype construction; we do not claim an expectation-maximization objective or a convergence guarantee. No labels from U_V enter these computations.

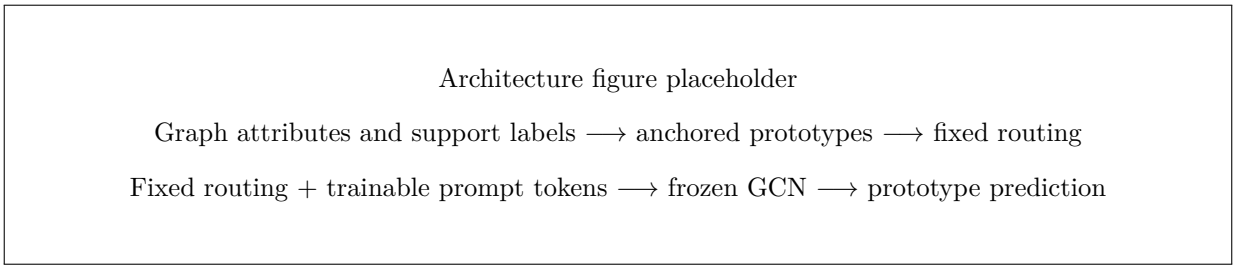


Figure 1: Overview of PRPrompt. Support labels initialize class references in aggregated attribute space. Unlabeled graph features refine these references, and the resulting node–prototype associations determine a fixed routing matrix. Only the class prompt tokens are optimized; the routed prompts enter the first hidden layer of a frozen GCN. Fitting and selection use only support labels, including overlapping test nodes under the original ProG split. The final architecture artwork is intentionally left as a placeholder.

4.2 From class relationships to a fixed routing matrix

Using the refined prototypes, we compute node–class associations at temperature $\tau_r = 0.1$:

$$\begin{aligned} Q_{ic} &= \mathrm{softmax}_c\Big(R_i^\top \,\mathrm{norm}\Big(\mu_c^{(3)}\Big)/\tau_r\Big), \\ B &= \widehat{A}\,(Q - \mathbf{1}\,\mathrm{mean}_{v\in V}\,Q_v)\,. \end{aligned} \tag{5}$$

Centering expresses a node's class association relative to the graph-wide average; neighborhood aggregation incorporates adjacent association patterns. Q is a row-normalized association matrix, whereas B contains signed prompt-combination coefficients. We do not interpret B as a class-probability matrix. B is computed once per support set and kept fixed throughout training.

4.3 Adapting frozen hidden computations

Let H^{pre} be the first GCN layer's preactivation on the original graph. We learn a zero-initialized matrix $P \in \mathbb{R}^{C\times 128}$. With s the vector of channelwise sample standard deviations of H^{pre}, clamped below by 10^{-6}, the prompted hidden representation is

$$\widetilde{H}^{\mathrm{pre}} = H^{\mathrm{pre}} + 10\,[B\tanh(P)] \odot s. \tag{6}$$

The scale vector is broadcast over nodes. The existing LeakyReLU and second graph convolution then produce Z. Thus, prompt content is learned through support supervision, while routing and backbone parameters remain fixed. Zero initialization recovers the frozen encoder at the start of optimization. The trainable prompt counts are 896, 768, and 384 for Cora, CiteSeer, and PubMed.

4.4 Training and prediction

We optimize the prompt matrix for 100 Adam steps with learning rate 0.01 and no weight decay:

$$\begin{aligned} \mathcal{L}(P) = \frac{1}{|S|}\sum_{i\in S} \mathrm{CE}(\ell_i(P), y_i) \\ + 10^{-3}\,\mathrm{mean}(P\odot P). \end{aligned} \tag{7}$$

The support logits ℓ_i use cosine prototype classification. The same prototype readout is kept throughout the routing ablation. Support leave-one-out construction, prediction normalization, and temperatures are specified in Appendix A.

The routing is a one-time computation. For sparse graphs, the two attribute aggregation steps cost $O(|E|d)$; prototype refinement costs $O(nCd)$ per iteration; and multiplying B by prompt tokens costs $O(nC\cdot 128)$ per optimization step, in addition to the frozen encoder's forward and backward computations. Freezing weights reduces trainable parameters but does not eliminate gradient computation through the second convolution. We report parameter counts, not an unmeasured runtime advantage.

5 Experiments

5.1 Protocol and comparison methods

The rerun covers eight datasets, with Cora, CiteSeer, and PubMed in the main text and the remaining five in Appendix D. We use the original ProG sampler: the first 90% of one random permutation forms the test set, while the last 10% of a second independent permutation forms the support pool. The first k members of each class are selected and shuffled. Each condition uses 20 task seeds and three training seeds. Tasks redraw the split; methods share support/test identities within a task, and shot-specific support sets are nested. Existing edge-pretrained checkpoints are reused without new pretraining. Support/test overlap is retained and recorded.

The local references are **Frozen**, with no prompt and the same final prototype readout as PRPrompt, and **Full FT**, updating the pretrained encoder and a task readout. Prompt baseline accuracy is taken from Tables 4–6 of Chen et al. [2]: GPPT, Gprompt (GraphPrompt), All-in-one, GPF, and GPF-plus, together with their supervised and fine-tuning references. Their MTG method is excluded. The source combines ProG results with author reproductions and reports best performance across pretraining strategies; the selected strategy is not identified for each cell.

Table 1: Accuracy on few-shot node classification. Higher is better (↑).

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Literature-reported baselines [2]									
Supervised	26.56± 5.55	37.79± 9.16	50.25± 8.37	21.78± 7.32	35.18± 6.86	41.22± 6.30	39.37± 16.34	57.33± 4.64	67.88± 2.18
Fine-tuning	52.61± 1.73	51.97± 2.84	62.66± 3.55	35.05± 4.37	45.08± 2.09	39.54± 3.54	46.74± 14.89	65.40± 3.00	**70.91**± 4.87
GPPT	43.15± 9.44	43.84± 6.11	51.98± 3.43	37.26± 6.17	42.34± 8.31	45.77± 7.41	<u>48.31</u>± 17.72	67.43± 2.96	66.97± 3.70
Gprompt	**56.66**± 11.22	**63.78**± 5.77	**69.03**± 3.61	<u>53.21</u>± 10.94	<u>60.00</u>± 6.18	<u>66.13</u>± 1.64	39.74± 15.35	66.68± 3.53	67.87± 2.08
All-in-one	52.39± 10.17	48.09± 4.83	30.36± 13.48	40.41± 2.80	48.09± 8.18	27.93± 10.59	45.17± 6.45	65.79± 5.79	46.16± 15.83
GPF	38.57± 5.41	34.84± 19.83	35.43± 1.02	31.16± 8.05	25.92± 12.30	25.12± 3.01	**49.99**± 8.86	**71.20**± 2.82	68.96± 3.99
GPF-plus	<u>55.77</u>± 10.30	<u>56.38</u>± 5.37	<u>66.22</u>± 6.20	**59.67**± 11.87	**72.48**± 5.63	**75.73**± 2.19	46.64± 18.97	<u>70.85</u>± 4.03	<u>69.59</u>± 4.33
Local measurements (different protocol)									
Frozen	24.64± 4.63	30.86± 3.27	35.16± 2.40	21.69± 3.47	24.96± 3.51	27.52± 3.72	<u>53.72</u>± 10.60	<u>62.49</u>± 9.18	<u>65.68</u>± 5.56
Full FT	<u>45.15</u>± 8.56	<u>59.29</u>± 6.26	<u>66.12</u>± 5.09	<u>33.59</u>± 5.82	<u>42.05</u>± 6.91	<u>49.92</u>± 5.85	49.90± 8.69	58.72± 8.45	64.14± 7.08
PRPrompt	**53.80**± 8.45	**69.97**± 3.53	**72.87**± 3.36	**43.92**± 11.01	**59.67**± 5.98	**64.70**± 3.53	**59.77**± 7.47	**65.09**± 8.21	**68.34**± 6.97

Mean ± std. (%). Literature: Chen et al. [2], Tables 4–6, five trials and best reported pretraining strategy. Local: reused checkpoint and original ProG sampling (90% test; independent support-pool draw, overlap retained); three training seeds averaged per task, then mean/sample std. over 20 tasks. **Bold**/<u>underline</u>: best/second within each panel and column. Literature split identities are unavailable; other settings differ; no cross-panel pairing or pooled average.

Table 2: Macro-F1 on few-shot node classification. Higher is better (↑).

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	23.79± 4.61	30.11± 2.90	34.48± 2.30	20.84± 3.15	23.85± 3.32	26.11± 3.34	<u>52.29</u>± 9.78	<u>61.43</u>± 9.05	<u>64.19</u>± 5.48
Full FT	<u>42.44</u>± 7.83	<u>58.11</u>± 6.07	<u>65.62</u>± 4.62	<u>28.52</u>± 6.08	<u>37.21</u>± 6.65	<u>45.91</u>± 5.39	46.50± 9.40	56.26± 10.15	61.97± 8.12
PRPrompt	**51.82**± 7.47	**68.37**± 4.08	**72.02**± 2.74	**41.48**± 10.31	**56.34**± 5.54	**61.34**± 3.09	**59.39**± 7.93	**64.82**± 8.80	**67.80**± 7.59

Local original-ProG runs with support/test overlap: mean ± sample std. over 20 tasks after averaging three training seeds per task. **Bold**/<u>underline</u>: best/second in each condition. No cross-condition average. Literature baselines do not report this metric in Tables 4–6.

Literature protocol. The source uses a 90% test allocation, five task samplings, and random hyperparameter search, mainly with a two-layer GCN of width 128. We match the original ProG split implementation, but do not have the source paper's actual split identities and cannot infer its overlap from upstream code. Checkpoints, selection, and repetition counts remain unmatched. Table 1 therefore separates literature and local panels, retaining source values without cross-panel pairing or ranking. The source tables report Accuracy only; other metrics are local measurements.

PRPrompt and the shared-prompt ablation use one fixed recipe. Local Full FT uses three support-only folds at 3 and 5 shots and four candidates: learning rate in $\{0.001, 0.01\}$ and weight decay in $\{0, 0.0001\}$. Each candidate fit and final fit uses 100 steps; 1-shot uses fixed parameters. Consequently, local methods do not have identical total optimization budgets. Appendix A gives the remaining settings.

Statistical unit. For a fixed local condition, we first average three training seeds within each task draw, then report the mean and sample standard deviation over the 20 task draws. We never average performance across datasets or shot counts. Local paired intervals resample task draws with 10,000 bootstrap samples. They are unadjusted 95% intervals for the fixed graph and checkpoint, and apply only to Frozen, Full FT, and shared-prompt comparisons. Accuracy and Macro-F1 are the primary reported metrics; the remaining local metrics appear in Appendix B.

5.2 RQ1: Adaptation performance

The local comparison tests adaptation under the same graph, checkpoint, and labeled support budget. Table 1 adds literature context with a separate protocol; Table 2 reports local Macro-F1. Each column is an individual condition, without a pooled average.

On Cora at 5 shots, PRPrompt reaches 72.87% accuracy compared with 35.16% for Frozen and 66.12% for Full FT. On CiteSeer at the same shot count, the corresponding accuracies are 64.70%, 27.52%, and 49.92%. PubMed has a stronger frozen reference: 65.68% at 5 shots, compared with 68.34% for PRPrompt and 64.14% for Full FT. Thus, the improvement over frozen pretraining is substantially smaller on PubMed, an important dependence that a cross-dataset average would hide.

Table 3: Prototype-routing ablation. Matched prompt recipe and readout.

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Accuracy (%) ↑									
Frozen	24.64± 4.63	30.86± 3.27	35.16± 2.40	21.69± 3.47	24.96± 3.51	27.52± 3.72	53.72± 10.60	62.49± 9.18	65.68± 5.56
Shared prompt	24.15± 4.41	31.25± 3.68	36.30± 2.72	20.64± 3.07	23.71± 3.88	25.60± 3.82	52.61± 10.70	60.89± 9.03	65.00± 5.72
PRPrompt	**53.80**± 8.45	**69.97**± 3.53	**72.87**± 3.36	**43.92**± 11.01	**59.67**± 5.98	**64.70**± 3.53	**59.77**± 7.47	**65.09**± 8.21	**68.34**± 6.97
Macro-F1 (%) ↑									
Frozen	23.79± 4.61	30.11± 2.90	34.48± 2.30	20.84± 3.15	23.85± 3.32	26.11± 3.34	52.29± 9.78	61.43± 9.05	64.19± 5.48
Shared prompt	22.78± 4.10	30.03± 3.70	34.69± 2.61	19.75± 2.74	22.28± 3.34	24.42± 3.38	51.98± 10.37	59.21± 10.06	63.59± 6.09
PRPrompt	**51.82**± 7.47	**68.37**± 4.08	**72.02**± 2.74	**41.48**± 10.31	**56.34**± 5.54	**61.34**± 3.09	**59.39**± 7.93	**64.82**± 8.80	**67.80**± 7.59

Local original-ProG runs with support/test overlap: mean ± sample std. over 20 tasks after averaging three training seeds per task. **Bold**/underline: best/second in each condition. No cross-condition average. Frozen: no prompt; shared prompt: uniform allocation; PRPrompt: prototype routing.

Literature-reported prompt results. The imported values do not support a claim of uniformly higher accuracy. On CiteSeer, GPF-plus reports 59.67, 72.48, and 75.73% at 1, 3, and 5 shots, whereas our corresponding measurements are 43.92, 59.67, and 64.70%. On Cora at 5 shots, our 72.87% lies above the reported Gprompt value of 69.03%; on PubMed at 3 shots, our 65.09% lies below the reported GPF value of 71.20%. These are descriptive cross-source comparisons, not matched estimates of method effects. The evidence for routing comes from the local ablation below; local paired differences are provided in Appendix C.

5.3 RQ2: Value of prototype routing

The central ablation replaces B in Eq. (6) by the constant matrix with entries $1/C$. This retains the prompt matrix, scale, injection point, optimization, and final readout, but removes node-dependent allocation. The constant matrix is used directly; centering it would erase the prompt. We also retain the no-prompt Frozen reference.

Table 3 shows that a shared prompt alone does not account for the gains. At 5 shots, routing improves accuracy over the shared prompt by 36.57 percentage points on Cora, 39.09 on CiteSeer, and 3.34 on PubMed. The respective paired 95% intervals are $[34.82, 38.17]$, $[37.37, 40.84]$, and $[1.03, 5.50]$ percentage points. The improvement is also positive in both primary metrics in each 1- and 3-shot condition.

This comparison supports the complete prototype-routing construction relative to a shared prompt. It does not isolate prototype refinement, centering, and graph aggregation as separate causes. A uniform mixture also reduces the effective degrees of freedom despite retaining the nominal prompt parameter count. We therefore do not interpret the comparison as a capacity-matched proof of a unique routing mechanism.

6 Limitations

The main text covers three citation graphs, with five further datasets in the appendix. Each uses one existing checkpoint, so pretraining-run variation is not estimated. Dataset scope was fixed before this rerun, but model development had already used these graphs; the analysis remains descriptive. Original ProG sampling allows support/test overlap, so reported scores include some labeled nodes and are not evidence of strictly held-out generalization. Some nominal 3/5-shot ogbn-arxiv tasks also lack the requested support count in a minority class. These limitations preclude broad claims about graph classification, unseen classes, or arbitrary pretraining sources.

The literature split identities are unavailable, with five trials and selection across pretraining strategies. Their aggregate values provide context but cannot isolate an adaptation-method effect against our fixed-checkpoint runs or support paired significance tests. Local fine-tuning selection also incurs a different budget from the fixed PRPrompt recipe. The routing ablation changes effective prompt capacity and does not distinguish every component of the routing construction. Finally, the method uses query features and graph-wide statistics; deployment without such transductive access requires separate evaluation.

7 Conclusion

PRPrompt uses support-anchored prototypes to allocate hidden prompts in a frozen graph encoder. Matched ablations on the three main datasets support its value over shared prompts; additional datasets expose fine-tuning advantages and weak-effect conditions. This conclusion is specific to the original ProG implementation protocol. Support/test overlap prevents these measurements from replacing a strictly held-out evaluation.

Broader Impact

The study concerns public graph node-classification benchmarks and does not evaluate decisions about individuals. Errors can propagate through graph aggregation and unlabeled prototype refinement, particularly when the graph or class structure differs from the evaluated setting. Applications involving consequential decisions require domain-specific evaluation beyond these benchmarks.

References

[1] Q. Chen, L. Wang, B. Zheng, and G. Song. DAGPrompT: Pushing the Limits of Graph Prompting with a Distribution-aware Graph Prompt Tuning Approach. In *Proceedings of the ACM on Web Conference 2025*, WWW '25, pages 4346–4358. ACM, Apr. 2025. doi: 10.1145/3696410.3714917. URL http://dx.doi.org/10.1145/3696410.3714917.

[2] Y. Chen, D. Ma, S. Zhang, Y. Liu, X. Lin, X. Zhao, W. Yang, W. Chen, and C. Zhou. Message Tuning Outshines Graph Prompt Tuning: A Prismatic Space Perspective. In T. Zhang, M. Dudik, M. Jaggi, A. Agarwal, S. Li, D. Schuurmans, J. Zhu, F. Berkenkamp, H. Dong, and A. Bietti, editors, *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 16063–16104. PMLR, 06–11 Jul 2026. URL https://proceedings.mlr.press/v306/chen26cz.html.

[3] T. Fang, Y. Zhang, Y. Yang, C. Wang, and L. Chen. Universal Prompt Tuning for Graph Neural Networks. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 52464–52489. Curran Associates, Inc., 2023. doi: 10.52202/075280-2285. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/a4a1ee071ce0fe63b83bce507c9dc4d7-Paper-Conference.pdf.

[4] W. Hamilton, Z. Ying, and J. Leskovec. Inductive Representation Learning on Large Graphs. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7ebea9-Paper.pdf.

[5] Z. Hou, X. Liu, Y. Cen, Y. Dong, H. Yang, C. Wang, and J. Tang. GraphMAE: Self-Supervised Masked Graph Autoencoders. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, pages 594–604. ACM, Aug. 2022. doi: 10.1145/3534678.3539321. URL http://dx.doi.org/10.1145/3534678.3539321.

[6] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec. Strategies for Pre-training Graph Neural Networks. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=HJlWWJSFDH.

[7] T. N. Kipf and M. Welling. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[8] J. Lee, W. Yang, and J. Kang. Subgraph-level universal prompt tuning. *Information Sciences*, 749:123516, Sept. 2026. ISSN 0020-0255. doi: 10.1016/j.ins.2026.123516. URL http://dx.doi.org/10.1016/j.ins.2026.123516.

[9] Z. Liang, L. Bai, and X. Yang. Attribute-guided Dynamic Prompt Learning for Graph Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(28): 23442–23450, Mar. 2026. ISSN 2159-5399. doi: 10.1609/aaai.v40i28.39515. URL http://dx.doi.org/10.1609/aaai.v40i28.39515.

[10] Z. Liu, X. Yu, Y. Fang, and X. Zhang. GraphPrompt: Unifying Pre-Training and Downstream Tasks for Graph Neural Networks. In *Proceedings of the ACM Web Conference 2023*, WWW '23, pages 417–428. ACM, Apr. 2023. doi: 10.1145/3543507.3583386. URL http://dx.doi.org/10.1145/3543507.3583386.

[11] Y. Ma, N. Yan, J. Li, M. Mortazavi, and N. V. Chawla. HetGPT: Harnessing the Power of Prompt Tuning in Pre-Trained Heterogeneous Graph Neural Networks. In *Proceedings of the ACM Web Conference 2024*, WWW '24, pages 1015–1023. ACM, May 2024. doi: 10.1145/3589334.3645685. URL http://dx.doi.org/10.1145/3589334.3645685.

[12] M. Ren, E. Triantafillou, S. Ravi, J. Snell, K. Swersky, J. B. Tenenbaum, H. Larochelle, and R. S. Zemel. Meta-Learning for Semi-Supervised Few-Shot Classification, 2018. URL https://arxiv.org/abs/1803.00676.

[13] J. Snell, K. Swersky, and R. Zemel. Prototypical Networks for Few-shot Learning. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/cb8da6767461f2812ae4290eac7cbc42-Paper.pdf.

[14] M. Sun, K. Zhou, X. He, Y. Wang, and X. Wang. GPPT: Graph Pre-training and Prompt Tuning to Generalize Graph Neural Networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, pages 1717–1727. ACM, Aug. 2022. doi: 10.1145/3534678.3539249. URL http://dx.doi.org/10.1145/3534678.3539249.

[15] X. Sun, H. Cheng, J. Li, B. Liu, and J. Guan. All in One: Multi-Task Prompting for Graph Neural Networks. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, pages 2120–2131. ACM, Aug. 2023. doi: 10.1145/3580305.3599256. URL http://dx.doi.org/10.1145/3580305.3599256.

[16] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph Attention Networks. *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=rJXMpikCZ. accepted as poster.

[17] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm. Deep Graph Infomax. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=rklz9iAcKQ.

[18] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen. Graph Contrastive Learning with Augmentations. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 5812–5823. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/3fe230348e9a12c13120749e3f9fa4cd-Paper.pdf.

[19] X. Yu, Y. Fang, Z. Liu, and X. Zhang. HGPrompt: Bridging Homogeneous and Heterogeneous Graphs for Few-Shot Prompt Learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(15):16578–16586, Mar. 2024. ISSN 2159-5399. doi: 10.1609/aaai.v38i15.29596. URL http://dx.doi.org/10.1609/aaai.v38i15.29596.

[20] X. Yu, C. Zhou, Y. Fang, and X. Zhang. MultiGPrompt for Multi-Task Pre-Training and Prompting on Graphs. In *Proceedings of the ACM Web Conference 2024*, WWW '24, pages 515–526. ACM, May 2024. doi: 10.1145/3589334.3645423. URL http://dx.doi.org/10.1145/3589334.3645423.

[21] X. Yu, J. Zhang, Y. Fang, and R. Jiang. Non-Homophilic

Graph Pre-Training and Prompt Learning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, KDD '25, pages 1844–1854. ACM, July 2025. doi: 10.1145/3690624.3709219. URL http://dx.doi.org/10.1145/3690624.3709219.

[22] J. Zhu, Z. Ding, J. Yu, J. Tan, X. Li, and W. Qian. RELIEF: Reinforcement Learning Empowered Graph Feature Prompt Tuning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, KDD '25, pages 2159–2170. ACM, July 2025. doi: 10.1145/3690624.3709252. URL http://dx.doi.org/10.1145/3690624.3709252.

[23] C. Zi, H. Zhao, X. Sun, Y. Lin, H. Cheng, and J. Li. ProG: A Graph Prompt Learning Benchmark. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 95406–95437. Curran Associates, Inc., 2024. doi: 10.52202/079017-3023. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/ad3e803a977f4279330c6ab7245937c6-Paper-Datasets_and_Benchmarks_Track.pdf.

A Reproducibility

A.1 Graph preprocessing and fixed checkpoints

Table 4 records the main dataset dimensions. Existing graph tensors and preprocessing are retained, but official masks are no longer used. Each task follows the original ProG implementation with two independent permutations for the 90% test set and 10% support pool. The first k members of each class are selected; other nodes provide unlabeled context. Support/test overlap is retained, with indices and counts stored in the result package.

Table 4: Dataset and prompt sizes.

Dataset	Nodes	Features	Classes	Prompt params.
Cora	2,708	1,433	7	896
CiteSeer	3,327	3,703	6	768
PubMed	19,717	500	3	384

The encoder consists of two GCNConv layers of width 128 with LeakyReLU between them. The first layer includes the pretrained affine transformation and graph aggregation; the prompt is added to its output before the nonlinearity. The second convolution has no additional activation in the evaluated encoder. The same checkpoint is used by every comparison method on a dataset. Checkpoint identity is checked at preparation, and task indices are retained. The recorded pretraining family is `Edgepred_Gprompt`; the pretraining random seed is unavailable.

A.2 Training and prediction details

The main text describes the active forward computation. For reproducibility, Adam uses $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, learning rate 0.01, and weight decay zero. The prompt regularizer is the mean of squared token entries multiplied by 10^{-3}. It is separate from optimizer weight decay. The channel scale uses the sample standard deviation, with denominator $n - 1$. Euclidean normalization uses the implementation's 10^{-12} norm floor. There is no distillation or additional test-label objective; overlapping test nodes receive supervision as support nodes.

For the support loss, let $\bar{z}_i = \mathrm{norm}(Z_i)$ and $m_c = \sum_{j \in S_c} \bar{z}_j$. When $\min_c k_c > 1$, the normalized prototype used for training support node i is

$$p_c^{(-i)} = \mathrm{norm}\left(\frac{m_c - \mathbb{I}[y_i = c]\bar{z}_i}{k_c - \mathbb{I}[y_i = c]}\right), \qquad \ell_{ic} = \bar{z}_i^\top p_c^{(-i)}/0.2. \tag{8}$$

When $\min_c k_c = 1$, the inclusive class prototype $\mathrm{norm}(m_c/k_c)$ replaces $p_c^{(-i)}$. During prediction, define

$$z_i^* = \mathrm{norm}\left(Z_i - \frac{1}{n}\sum_{j \in V} Z_j\right), \qquad p_c^* = \mathrm{norm}\left(\frac{1}{k_c}\sum_{i \in S_c} z_i^*\right), \qquad \widehat{y}_i = \arg\max_c (z_i^*)^\top p_c^*. \tag{9}$$

The probability scores for all five metrics are obtained by applying softmax to cosine logits divided by 0.2. Training does not apply the output centering used at prediction. The readout is a fixed implementation choice, not an experimental factor in this paper.

Procedure. Given a graph, a fixed checkpoint, and support labels:

1. Normalize adjacency and compute Eq. (1) and the first-layer preactivation.
2. Initialize support prototypes with Eq. (2) and apply Eqs. (3)–(4) three times.
3. Compute the fixed route with Eq. (5); for the shared-prompt ablation, replace it by $B_{ic} = 1/C$.
4. Initialize $P = 0$ and cache the hidden-channel scales. Keep the encoder and route fixed.
5. For 100 steps, run the prompted encoder, form support logits, and update P using Eq. (7).
6. Compute final prototype scores using Eq. (9); seal predictions before scoring with query labels.

A.3 Baseline fitting and seed control

Task seeds are 30093000 through 30093019 and training seeds are 0,1,2. Each task first draws test nodes, then independently draws the support pool, selects the first k nodes per class, and shuffles support order. Methods share task identities and shot-specific support sets are nested. Insufficient classes retain $k_c < k$ without replenishment. The no-training Frozen method retains three seed slots for alignment; these are not independent fits.

Local Full FT uses support cross-entropy and a task readout. Its parameters when the minimum class support count is one are learning rate 0.01 and weight decay 0.0001. When the minimum count exceeds one, each of $\min(3, \min_c k_c)$ support-only folds holds out one node per class. Four candidate configurations are ranked by mean held-out support cross-entropy, then Full FT is refit on the complete support set. No external validation-label budget is used. This local protocol controls labeled information and final step count, but not total selection compute or readout parameterization. It does not describe the imported literature baselines.

A.4 Condition-specific statistics

For a fixed dataset, shot count, checkpoint, and method, let a_{st} denote a metric with task seed s and training seed t. We

compute

$$\bar{a}_s = \frac{1}{3}\sum_{t=1}^{3} a_{st}, \qquad \widehat{a} = \frac{1}{20}\sum_{s=1}^{20} \bar{a}_s,$$
$$\widehat{\sigma} = \sqrt{\frac{1}{19}\sum_{s=1}^{20}(\bar{a}_s - \widehat{a})^2}. \tag{10}$$

Local cells report $100\widehat{a} \pm 100\widehat{\sigma}$, not the standard error. For local paired comparisons, $d_s = \bar{a}_s^{\mathrm{PRPrompt}} - \bar{a}_s^{\mathrm{reference}}$ is resampled as a task-draw block. The percentile bounds of 10,000 resampled means form the reported 95% interval. All intervals are recomputed for this rerun. Literature cells retain their source five-trial mean and standard deviation; Eq. (10) does not apply to them. No metric average or mean effect is computed across datasets, shots, methods, or sources.

Accuracy, Macro-F1, Micro-F1, macro one-vs-rest ROC-AUC, and macro average precision are retained in the data package. In this single-label multiclass setting, Micro-F1 coincides with Accuracy; it is included for completeness. Support and test sets are redrawn across tasks; intervals reflect task sampling and training variation conditional on the graph, not independent graph samples.

A.5 Environment, provenance, and numerical qualification

The existing ProG environment is reused with PyTorch 2.11.0+cu130 and the same graph inputs and checkpoints. Per-unit fitting times are recorded; concurrent wall-clock times do not establish a method-level efficiency advantage.

The eight-dataset rerun completed 5760 formal units and 96 separate two-step smoke units. Local checks cover coverage, finite metrics, paired support/test indices, and recomputed means, standard deviations, and bootstrap intervals. Saved-embedding readout uses a 2×10^{-6} tolerance. An old strict filter row-sum assertion was converted to error recording without changing operator arithmetic. No exhaustive file-hash scan or fresh CUDA bitwise audit was repeated; earlier numerical audits are not presented as evidence for this rerun.

External Accuracy values are transcribed from Chen et al. [2], Tables 4–6 (PDF pages 37–38 of arXiv:2606.03290v1). The record preserves table/page identifiers, decimal strings, source URL and PDF hash. It contains 147 node-classification cells excluding MTG; the manuscript uses 63 cells for its three datasets. Source tables combine ProG results and author reproductions, without cell-specific attribution or selected pretraining identifiers. Raw predictions are unavailable in the imported record, so no external paired tests are computed. Local historical prompt measurements remain archived but are not substituted for missing literature metrics.

Code and results are available at https://github.com/Mes-sala/PRPrompt; execution scripts are in `experiments/prog_split_20261002`. This work reruns adaptation and statistics under original ProG sampling without new pretraining. The first seven datasets reuse their original Edgepred_Gprompt checkpoints with unknown pretraining seeds. ogbn-arxiv reuses its previously trained full-graph edge-prediction checkpoint with pretraining seed 42. Weights, graph snapshots, and model states remain on the server; local artifacts contain code, predictions, splits, tables, and documents.

B Additional Metrics

The following tables extend the main comparison with the remaining stored metrics under the same seed aggregation rule. Every column is a separate dataset/shot condition.

Table 5: Micro-F1 on few-shot node classification. Higher is better (↑).

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	24.64± 4.63	30.86± 3.27	35.16± 2.40	21.69± 3.47	24.96± 3.51	27.52± 3.72	53.72± 10.60	62.49± 9.18	65.68± 5.56
Full FT	45.15± 8.56	59.29± 6.26	66.12± 5.09	33.59± 5.82	42.05± 6.91	49.92± 5.85	49.90± 8.69	58.72± 8.45	64.14± 7.08
PRPrompt	**53.80**± 8.45	**69.97**± 3.53	**72.87**± 3.36	**43.92**± 11.01	**59.67**± 5.98	**64.70**± 3.53	**59.77**± 7.47	**65.09**± 8.21	**68.34**± 6.97

Local original-ProG runs with support/test overlap: mean ± sample std. over 20 tasks after averaging three training seeds per task. **Bold**/underline: best/second in each condition. No cross-condition average. Literature baselines do not report this metric in Tables 4–6.

Table 6: Macro ROC-AUC on few-shot node classification. Higher is better (↑).

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	63.21± 4.73	69.93± 2.36	73.21± 2.16	56.08± 3.35	59.27± 3.49	61.37± 2.97	71.26± 10.76	80.38± 8.27	83.07± 4.42
Full FT	79.66± 4.17	88.47± 3.03	91.72± 2.18	69.34± 5.25	74.01± 3.87	78.59± 3.00	71.68± 7.97	79.93± 5.25	**83.87**± 4.51
PRPrompt	**81.14**± 5.37	**90.82**± 2.12	**92.22**± 1.34	**72.42**± 7.93	**82.11**± 2.86	**84.12**± 2.17	**77.21**± 6.53	**81.72**± 7.67	83.84± 5.88

Local original-ProG runs with support/test overlap: mean ± sample std. over 20 tasks after averaging three training seeds per task. **Bold**/underline: best/second in each condition. No cross-condition average. Literature baselines do not report this metric in Tables 4–6.

Table 7: Macro AP on few-shot node classification. Higher is better (↑).

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	24.76± 4.19	30.07± 2.96	34.33± 2.50	21.33± 2.42	23.68± 2.88	25.30± 3.00	55.51± 10.07	66.73± 9.18	68.93± 6.61
Full FT	50.83± 7.37	67.63± 5.63	75.34± 4.11	36.52± 6.95	43.52± 6.52	50.83± 5.30	56.10± 9.47	66.58± 7.45	**72.23**± 6.16
PRPrompt	**56.27**± 8.10	**74.43**± 4.49	**78.20**± 2.98	**42.17**± 10.92	**57.10**± 5.18	**61.42**± 3.29	**61.55**± 8.90	**68.29**± 9.31	70.78± 8.65

Local original-ProG runs with support/test overlap: mean ± sample std. over 20 tasks after averaging three training seeds per task. **Bold**/underline: best/second in each condition. No cross-condition average. Literature baselines do not report this metric in Tables 4–6.

C Paired Differences and Uncertainty

Each cell reports PRPrompt minus a local reference in percentage points, with its unadjusted 95% paired task-bootstrap interval. All entries preserve original paired statistics. Literature baselines are excluded because the runs are not paired. Tables group conditions for readability but do not average them.

Table 8: Paired Accuracy gains of PRPrompt. Differences in percentage points.

Reference	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	+29.16 [25.55, 32.58]	+39.11 [37.04, 40.76]	+37.72 [36.15, 39.08]	+22.23 [18.15, 26.00]	+34.70 [32.66, 36.60]	+37.18 [35.66, 38.74]	+6.05 [1.54, 10.63]	+2.61 [-2.17, 6.99]	+2.66 [-0.33, 5.15]
Full FT	+8.64 [6.74, 10.55]	+10.68 [8.82, 12.81]	+6.75 [5.24, 8.27]	+10.33 [7.68, 12.85]	+17.61 [15.55, 19.53]	+14.78 [12.47, 17.05]	+9.87 [7.35, 12.44]	+6.37 [2.84, 10.10]	+4.20 [1.43, 7.07]
Shared prompt	+29.65 [26.14, 32.91]	+38.72 [36.85, 40.43]	+36.57 [34.82, 38.17]	+23.27 [19.38, 27.21]	+35.96 [33.80, 38.08]	+39.09 [37.37, 40.84]	+7.16 [2.85, 11.74]	+4.20 [0.77, 8.15]	+3.34 [1.03, 5.50]

PRPrompt minus the reference (top); unadjusted 95% paired task-bootstrap interval (below), recomputed for the original-ProG rerun. Support/test overlap is retained.

Table 9: Paired Macro-F1 gains of PRPrompt. Differences in percentage points.

Reference	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	+28.02 [24.70, 31.23]	+38.26 [36.09, 40.03]	+37.53 [36.21, 38.79]	+20.64 [16.82, 24.26]	+32.48 [30.52, 34.29]	+35.23 [33.82, 36.64]	+7.10 [2.43, 11.61]	+3.39 [-1.69, 7.90]	+3.60 [0.43, 6.17]
Full FT	+9.38 [7.57, 11.12]	+10.26 [8.42, 12.40]	+6.39 [4.85, 7.92]	+12.96 [10.44, 15.41]	+19.13 [17.02, 21.23]	+15.42 [13.31, 17.53]	+12.88 [9.62, 16.15]	+8.56 [4.78, 12.71]	+5.82 [2.23, 9.61]
Shared prompt	+29.04 [26.04, 31.84]	+38.34 [36.08, 40.43]	+37.32 [35.82, 38.86]	+21.73 [18.11, 25.42]	+34.05 [32.06, 35.95]	+36.91 [35.31, 38.49]	+7.41 [2.92, 12.11]	+5.60 [2.16, 9.66]	+4.21 [1.98, 6.33]

PRPrompt minus the reference (top); unadjusted 95% paired task-bootstrap interval (below), recomputed for the original-ProG rerun. Support/test overlap is retained.

D Additional Datasets

Photo, Computers, Actor, Chameleon, and ogbn-arxiv use the same task-level statistics. All four methods are shown without averaging datasets. Original ProG sampling is retained. On ogbn-arxiv, seven nominal 3-shot and thirteen nominal 5-shot tasks lack the requested number of supports in a minority class, with actual k_c between 1 and 4. Four shortage tasks have minimum count one and retain the inclusive prototype loss; otherwise LOO uses actual class counts. Pools are not replenished.

Table 10: Accuracy on additional datasets.

Dataset	Shot	Frozen	Full FT	Shared prompt	**PRPrompt**
Photo	1	47.73± 5.38	50.87± 6.88	34.10± 4.92	**60.96**± 8.88
	3	58.82± 4.41	**74.78**± 5.52	49.55± 7.34	72.96± 4.71
	5	63.09± 4.16	**80.68**± 3.32	52.46± 6.11	73.10± 5.28
Computers	1	38.13± 6.59	39.95± 9.06	33.65± 6.68	**45.60**± 8.85
	3	47.56± 3.76	**62.19**± 5.57	46.29± 6.22	57.53± 7.03
	5	52.41± 2.89	**68.79**± 4.47	51.29± 3.53	63.12± 3.12
Actor	1	20.61± 1.22	20.28± 3.19	20.20± 1.59	**20.64**± 2.30
	3	21.14± 1.46	20.87± 1.86	20.42± 1.27	**21.24**± 1.40
	5	21.12± 1.63	20.97± 1.80	20.98± 1.21	**21.34**± 1.69
Chameleon	1	21.47± 3.01	20.91± 2.12	22.22± 3.11	**22.90**± 3.53
	3	22.85± 2.73	24.00± 3.44	23.42± 2.76	**25.09**± 4.10
	5	23.71± 3.05	26.05± 3.35	24.03± 3.34	**27.29**± 3.80
ogbn-arxiv	1	10.93± 1.66	**16.28**± 3.79	10.57± 1.80	11.07± 1.67
	3	16.13± 2.03	**21.67**± 3.28	18.06± 2.22	19.97± 2.26
	5	19.54± 1.99	**24.03**± 2.57	22.52± 2.23	23.95± 1.94

Local original-ProG runs with support/test overlap: mean ± sample std. over 20 tasks after averaging three training seeds per task. **Bold**/underline: best/second in each condition. No cross-condition average. For ogbn-arxiv, shots are nominal: 7/20 three-shot and 13/20 five-shot tasks have a class shortage; see text.

Table 11: Macro-F1 on additional datasets.

Dataset	Shot	Frozen	Full FT	Shared prompt	**PRPrompt**
Photo	1	48.13± 4.86	50.48± 6.11	33.00± 4.70	**60.54**± 7.59
	3	59.13± 3.63	**73.90**± 4.73	49.41± 6.56	71.58± 3.59
	5	62.98± 3.93	**79.62**± 2.74	51.20± 5.62	72.32± 4.45
Computers	1	36.39± 5.76	38.66± 8.09	31.90± 5.77	**44.77**± 7.13
	3	44.93± 3.70	**61.51**± 5.38	43.38± 5.89	54.79± 5.46
	5	48.50± 3.17	**68.65**± 3.88	46.89± 4.24	59.44± 2.98
Actor	1	19.12± 0.98	16.16± 2.35	**19.21**± 0.98	19.00± 1.64
	3	19.23± 1.62	18.45± 1.20	19.65± 0.90	**20.06**± 0.97
	5	19.44± 1.57	19.10± 0.98	20.16± 0.93	**20.49**± 1.51
Chameleon	1	20.38± 2.89	14.27± 2.60	19.87± 3.30	**20.72**± 3.94
	3	22.04± 2.64	19.98± 3.68	21.55± 3.10	**23.32**± 3.93
	5	22.75± 3.02	23.10± 3.33	22.00± 3.37	**26.03**± 3.39
ogbn-arxiv	1	8.35± 1.08	**9.95**± 1.45	7.82± 0.98	7.92± 1.05
	3	12.77± 1.08	**14.46**± 1.25	13.08± 1.18	14.16± 1.05
	5	15.59± 1.01	16.68± 1.17	16.44± 1.08	**17.21**± 0.99

Local original-ProG runs with support/test overlap: mean ± sample std. over 20 tasks after averaging three training seeds per task. **Bold**/underline: best/second in each condition. No cross-condition average. For ogbn-arxiv, shots are nominal: 7/20 three-shot and 13/20 five-shot tasks have a class shortage; see text.

Full fine-tuning outperforms PRPrompt on Photo and Computers at 3/5 shots, and the ogbn-arxiv 1-shot gap is particularly pronounced. Several Actor routing-effect intervals include zero; its 1-shot Macro-F1 mean difference from the shared prompt is negative. The 1-shot routing effects on Chameleon and ogbn-arxiv are also uncertain. These results do not establish universal superiority across graphs. All five metrics, paired intervals, and actual class-support counts remain in the data package.